LevelDB is not an SQL database and doesn't have a relational data model. It doesn't support SQL queries or indices. Applications make use of LevelDB as a library, and it doesn't provide a server or command-line interface.

LevelDB is much better than SQLite and Kyoto Cabinet in terms of read and write operations, and also the best in batch writes. It also performs better in terms of read, write and synchronisation-based operations when compared to Berkeley DB and OpenLDAP Lightning DB.

LevelDB is written in C++. iOS developers can use it directly in their applications or through one of the several Objective-C wrappers that are available for it. Android developers can use LevelDB via JNI and NDK.

Features

- Makes multiple operations atomic.
- Creates consistent snapshots.
- Is iterative over key ranges.
- Offers automatic data compression using the Snappy Compression library.
- *Basic operations: Put* (key, value), *get* (key) and *delete* (key)
- Facilitates transient snapshots for a consistent view of the data

Official website: *http://leveldb.org/*
Latest version: 1.14

UnQLite

UnQLite is an open source embedded NoSQL database engine designed in 2012 by Mrad Chems Eddine and his colleagues while working on distributed P2P and VoIP solutions like Skype. Its key-value storage operations are similar to Berkeley DB and MongoDB, and it has an inbuilt scripting language called Jx9, which is like JavaScript.

The main design objective of UnQLite is to store information of all nodes like the IP-address, blobs, etc. UnQLite was created by considering SQLite3 as the backend, i.e., the VFS layer, locking mechanism and Transaction Manager with Jx9.

UnQLite architecture: UnQLite is suitable for embedded devices and is written in ANSI C. It is thread safe, fully re-entrant and compiles unmodified. It is available for various platforms like Windows, FreeBSD, Solaris, Mac and for mobile platforms.

Features

- JSON document store via Jx9
- Fully ACID-compliant
- Pluggable run-time interchangeable storage engine
- Simple, clean and easy-to-use API
- BSD licensed
- Key-value store
- Supports terabyte-sized databases

Official website: *https://unqlite.org/*
Latest version: 1.18

Firebase

The Firebase real-time database is cloud hosted. Data is stored as JSON and synchronised in real-time to every connected client. On building cross-platform apps with iOS, Android and JavaScript, all clients share one real-time database instance and automatically receive updates of new data.

Features

- *Real-time, fast synchronisation:* Firebase uses data synchronisation—every time data changes, any connected device receives that update within milliseconds.
- *Offline access:* Firebase apps remain responsive even when offline because the Firebase SDK persists your data to disk. Once connectivity is re-established, the client device receives any changes it missed, synchronising it with the current server state.
- *Accessible via client devices:* The Firebase real-time database can be accessed directly from a mobile device or Web browser; there's no need for an application server. Security and data validation are available through its security rules, which are expression-based rules that are executed when data is read or written.

Official website: *https://firebase.google.com/*

References

[1] *https://www.sqlite.org/*
[2] *http://ormlite.com/*
[3] *http://www.oracle.com/technetwork/database/database-technologies/berkeleydb/overview/index.html*
[4] *http://objectbox.io*
[5] *https://realm.io/*
[6] *https://www.couchbase.com/products/lite*
[7] *http://leveldb.org/*
[8] *https://unqlite.org/*
[9] *https://firebase.google.com/*

By: Dr Anand Nayyar

The author is a professor at Duy Tan University in Vietnam. He loves to work on open source technologies, embedded systems, sensor networks, cloud computing, simulators, networks and ethical hacking. He can be reached at *anand_nayyar@yahoo.co.in*. YouTube channel: Gyaan with Anand Nayyar at *www.youtube.com/anandnayyar*.